LicenseToken-2026: Reference Implementation and Validation

Quantum Discoveries
quantumdiscoveries@proton.me

Abstract

A whitepaper describes what a system should do. This document records what the system actually does. It is the empirical companion to *LicenseToken-2026: On-Chain Software Licensing as a Native Asset*, and it exists so that the claims of that paper can be checked rather than trusted. The protocol described there is implemented, and the implementation is exercised at three levels: in isolation, inside the Solana runtime against the real compiled program, and against a live validator over the network as an ordinary client would reach it. Every claim that can be tested is tested. Where a property cannot be proven before launch, this document says so plainly. The summary is simple: the protocol compiles, deploys, charges, settles, and enforces exactly as the whitepaper states, and a second, unrelated program adopts it with two lines of code and is gated end to end on a live validator.

---

1. What Is Implemented

The reference implementation consists of three parts.

The first is the protocol program itself, which holds the entire grammar of the whitepaper: configuration with flat and tiered pricing, minting, renewal, expiry with grace, gift, atomic sale with royalty, revocation, supply caps, the permanent founder split, the separation of upgrade authority from licensing authority, and renouncement in degrees. It takes no fee and holds no privileged account.

The second is the integration layer: two procedural macros, `#[licensed_program]` and `#[gated]`, that let any Anchor program adopt licensing by changing one word and tagging the instructions it wishes to gate. The verification described in the whitepaper is compiled into the integrator's own program; there is no call back to the protocol at the moment of the check.

The third is two consumer programs that use the protocol as a real developer would. The first, `example-gated`, exercises every access mode. The second, `premium-counter`, is deliberately unrelated to the first — a program that maintains an on-chain counter — and exists to prove the integration is portable rather than tuned to a single example. Its entire dependence on the protocol is the two macros.

2. How It Is Tested

Testing proceeds at three levels, each stronger evidence than the last.

The first level is the protocol in isolation: its arithmetic, its state transitions, and its rejections, run as ordinary unit and integration tests.

The second level runs the **real compiled program inside the Solana virtual machine.** This is not a mock and not a simulation of behavior; it loads the same bytecode that would be deployed and executes it under the same runtime rules, including rent, account ownership, and instruction dispatch. Both the protocol and the consumer programs are loaded together, so a license minted by one program is checked by another exactly as it would be on-chain.

The third level is a **live validator.** A running node is started, the compiled programs are deployed to it with real transactions, and the protocol is then driven over JSON-RPC, the same interface a wallet or a web client uses. Transactions are signed, sent, confirmed, and their effects read back from chain state. Nothing about this path is privileged; it is the ordinary way software reaches Solana.

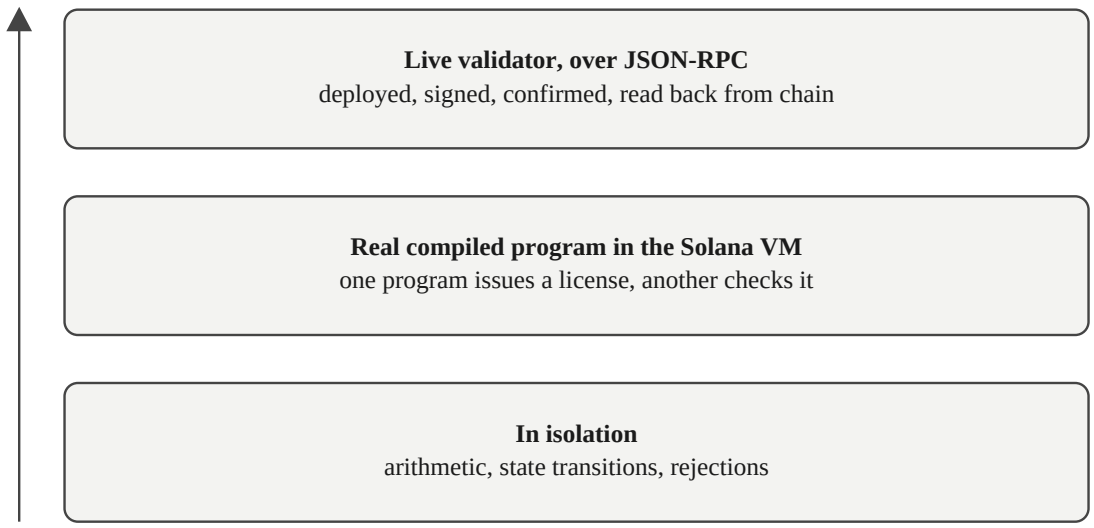

Figure 7. Three levels of evidence, each stronger than the last. The same program is checked in isolation, run as real bytecode inside the Solana virtual machine, and finally driven on a live validator exactly as a wallet would reach it.

3. Results

The automated suite is eighty-nine tests, all passing.

```
protocol behavior (mint, renew, sell, tiers,         37 tests
  founder split, renounce, supply caps, expiry)
adversarial (spoofed config, wrong program,           8 tests
  authority bypass, grace abuse)
boundary & edge cases (tier lifecycle, founder       23 tests
  on renewal and sale, pricing, expiry edges,
  per-tier caps, arithmetic overflow)
integration — example-gated (every access mode)      13 tests
integration — premium-counter (a second program)      7 tests
library                                               1 test
-------------------------------------------------------------
total                                                89 tests, 0 failures
```

The boundary suite is a deliberate effort to find leaks. Every error the protocol can return is triggered by a test that proves it fires when it should, with two exceptions that are unreachable by construction: an arithmetic path the runtime's own limits prevent from being approached except at the extreme tested here, and a malformed-program check the runtime never lets an account violate. Beyond the error codes, the claims most easily left untested are tested directly: a retired tier still lets its holders renew, a paused tier freezes both, the founder's share is taken on renewals and on sale royalties and not only on the first sale, renewal is charged at the current price, and a license is rejected at the exact second it expires.

To these is added one live-validator flow, run on demand: seven state-changing transactions that confirm on-chain — create a configuration, initialize, mint a license, increment under a license, increment as the creator, revoke the license, and reset as the authority — and three access attempts that the node rejects, including a holder whose license was revoked one transaction earlier, with every result read back from the chain. It is held separate because, unlike the others, it requires a running node.

4. Economics, Verified

The whitepaper states that payment settles in full to the creator, to the last lamport, with no protocol fee. The live flow tests this directly. A buyer pays the configured price of one million lamports for a license, and the treasury balance is read from chain before and after.

```
treasury delta = 1,000,000 lamports   (expected 1,000,000)
```

The amount received is exactly the price set, with nothing withheld. The protocol takes none of it, because there is no protocol account for it to take into.

5. Enforcement, Verified

The whitepaper states that a program consults the license before its privileged logic, and that the check defends itself by rebinding the configuration to its canonical address. Both are observed on the live validator.

A license holder calls the gated instruction and the counter advances. A stranger, presenting another wallet's license, is rejected and the counter does not move. The creator, holding no license, passes by the authority exception. A non-authority attempting an authority-only instruction is rejected. A license

minted for a different program does not unlock this one. And a license the authority revokes stops working in the very next transaction: the holder who advanced the counter moments earlier is denied the instant their license is revoked. Each rejection is a real failed transaction, not an assertion in a harness.

The chain reports each rejection precisely. A denied caller receives the gate verdict together with the specific cause, written to the program log:

```
Program log: LicenseToken gate denied: AnchorError { error_name: "NotHolder",
  error_code_number: 6004, error_msg: "Caller is not the license holder",
  error_origin: ...src/verify.rs, line: 64 }
Program log: AnchorError caused by account: license. Error Code:
  Unauthorized. Error Number: 6026.
```

The reason, its code, its message, and even its source line are present in the transaction a client already receives. Enforcement is not advisory: a transaction that fails the check does not execute, and its failure is legible.

6. Composability, Verified

The whitepaper claims a license can gate any program. The `premium-counter` program is the test of that claim. It shares no code with the first example. Its adoption of the protocol is, in full:

```
#[licensed_program]          // replaces #[program]
pub mod premium_counter { ... }

#[gated]                     // on the instruction's accounts
pub struct Increment<'info> { ... }
```

It compiled, deployed to the live validator, and gated correctly on the first attempt. A primitive earns the name by working on programs its author never saw; this is that test, passed.

7. Persistence, and a Note on Verification

A claim of on-chain execution should be checkable by the reader, not asserted by the author. The strongest such evidence is not a transaction signature, which a test validator prunes from its history within minutes, but the **account state those transactions created, which persists.** The configuration account for the consumer program can be read directly from the chain:

```
Owner: DKyyHjfzqSGjD1Z8dcBsjrrkdt88MuyaoBrhNQn9BQP8   (the protocol program)
Length: 907 bytes
Executable: false
```

An account owned by the protocol program can be created only by the protocol program. Its existence is proof that the configuration instruction ran. State cannot exist without the transaction that wrote it. This is the same standard of evidence the chain offers anyone: not a record of a promise, but the present, readable consequence of an act.

8. What Is Not Claimed

This document proves that the protocol functions as designed under real execution. It does not, and cannot, prove adoption, market demand, or economic outcome; no implementation can prove those before it is used. It does not extend the protocol's reach beyond its stated scope. As the whitepaper states, the enforcement is complete for on-chain programs, where the runtime performs the check as a condition of running, and it does not reach software the chain does not execute. The tests here are confined to what the protocol claims to do, and within that boundary they are exhaustive.

The validator here is a local node running the Solana runtime — the same runtime a public cluster runs, reached the same way. Deployment to a public cluster, devnet or mainnet, is a separate step, and this document does not claim it.

One implementation note belongs in the open. The canonical-address binding that defends the check is computed on every gated instruction; it uses the bump recorded at the configuration's creation, so the check is a single hash rather than a search, which keeps the cost low enough for high-frequency programs. The behavior is identical to a full re-derivation; only the cost differs.

9. Reproducing This

Every result above is produced by the project's own test suite — the same code that implements the protocol, not a separate harness written to flatter it. The isolated and in-runtime tests run with a single command and require nothing external. The live-validator flow requires a running node with the programs deployed, after which it runs the same way. These tests are the means by which the results were obtained, and they produce them again on every run.

The protocol described in the whitepaper is not a proposal awaiting implementation. It is implemented, tested at three levels, deployed to and exercised on a live validator, and observed to charge, settle, and enforce exactly as written. The fourth asset type is not only specified. It runs.